

Service Desk — Features and Processes

Version: 1.0 | **Last Updated:** February 2025 | **Document Type:** Feature and Process Documentation (Detailed)

Table of Contents

1. [Ticket Management](#)
 2. [Template System](#)
 3. [Approval System](#)
 4. [SLA Management](#)
 5. [Asset Management](#)
 6. [Knowledge Base](#)
 7. [AI FAQ Chat](#)
 8. [Organizations and Teams](#)
 9. [Departments and Hierarchy](#)
 10. [Users and Authentication](#)
 11. [Reports and Analytics](#)
 12. [Dashboard](#)
-

1. Ticket Management

A ticket is the central unit of work in the Service Desk. It represents a single support request—whether an incident (something broken), a service request (something needed), a problem (underlying cause), or a change (planned modification). Every ticket has a unique ticket number (for example SD-20250210-00001), a subject, a description, and is classified by status, priority, type, and channel. The system enforces a clear lifecycle: tickets start as New, move through Open, and can be set to Pending or On Hold while waiting on the customer or another team. When work is done, an agent marks the ticket Solved; once the requester confirms or time passes, it can be Closed. Requester and agents can reopen a ticket from Solved or Closed back to Open, and the system tracks how many times a ticket was reopened as a quality metric.

You can create tickets in two ways: manually or from a template. Manual creation is the classic flow: you open the “New ticket” dialog, choose subject and description, set priority (Low, Normal, High, Urgent), type (Incident, Service Request, Problem, Change), and channel (Portal, Email, Phone, Chat, API). If you are an agent or admin, you can also set the organization, team, and assignee up front. The ticket is then created with status New and, if SLA is configured, response and resolution due dates are calculated automatically from the ticket’s priority and team. Template-based creation is the alternative: you first choose “From Template,” then pick a template from the template gallery (filtered by category or search). The form you see is built dynamically from that template’s custom fields—text, dropdowns, dates, file uploads, and so on—plus the standard ticket fields (subject, description, priority, type, team, etc.) as defined in

the template. When you submit, the system creates the ticket and stores the values you entered as the “template response” so they can be viewed later on the ticket. If the template has an approval flow configured, the ticket may immediately enter an approval workflow (see Approval System).

Once a ticket exists, agents and admins can work with it on the ticket detail page. The ticket shows its current status, priority, assignee, and primary team. You can change status using the workflow (for example New → Open → Solved → Closed). The system only allows valid status transitions: for instance you cannot jump directly from Closed to Solved without reopening to Open first. Assignment is flexible: a ticket can have one assignee (the person responsible) and one primary team. There is also multi-team support: you can attach additional teams to a ticket, and for each team you can mark that team’s work as complete or reopen it, so several teams can contribute and track their part of the work independently.

Collaboration happens through comments. Comments can be public (visible to the requester) or private (internal notes for agents only). Each comment is stored with its author and timestamp. The first public comment from an agent is often used to mark that the “first response” SLA has been met. The ticket also has a full audit trail: every change to status, priority, assignee, team, or other key fields is recorded as an event with the previous and new values, so you can see exactly who changed what and when. Tags can be added to tickets for filtering and grouping (for example “billing,” “vip,” “hardware”). Attachments are supported so requesters and agents can attach files to the ticket; the system stores file name, type, size, and storage path.

Quick actions help agents work faster: “Assign to me” assigns the ticket to the current user, and “Escalate” raises the priority by one level (for example Normal to High) and logs an event. Bulk operations allow updating status, priority, assignee, or team for multiple tickets at once. Reassignment can also be done through a dedicated reassign action where you choose the new assignee and optionally the new team. All of this ties into SLA: when a ticket is created or when its priority or team changes, the system finds the best-matching SLA policy and sets or updates the response and resolution due dates for that ticket.

2. Template System

The template system lets administrators define reusable request types so that end users and agents do not have to start from a blank form every time. A template describes both what fields the user must fill and what default values the resulting ticket gets (subject, priority, type, team, assignee, organization). Templates are organized into categories, and each template can have an optional multi-step approval flow. The main parts of the system are: template categories, the template builder (including the form builder and approval flow module), the template gallery used when creating tickets, and the way template responses are stored and displayed on tickets.

2.1 Template Categories

Categories group templates for easier browsing and management. Each category has a name, an optional description, an icon (for example clipboard, file-text, settings), a sort order, and an

active flag. In the Template Builder page, admins can create, edit, reorder, and deactivate categories. When a user opens the template gallery to create a ticket, they can filter templates by category so that only relevant request types (e.g. "Access Request," "Hardware," "Software") are shown. Categories do not restrict which team or approval flow a template uses; they are purely for organization and discovery.

2.2 The Template Builder and Form Builder

The Template Builder is the admin interface for creating and editing templates. A template has basic metadata: name, description, category, and icon. The core of the template is the form schema: a list of fields that will be shown when someone creates a ticket from this template. This schema is edited in the Form Builder, which is a drag-and-drop designer. The admin adds fields from a toolbox and configures each field in a properties panel. The supported field types are: text (single line), textarea (multi-line), richtext (rich text), select (single choice from options), multiselect (multiple choices), checkbox_group, radio (single choice), number, date, daterange (start and end date), file_upload (file attachment), user_lookup (select a user from the system), section_header (heading to group fields), info_text (read-only text or instructions), divider (visual separator), and hidden (value stored but not shown). Each field has a unique id, a label, optional placeholder, required flag, and for select-type fields a list of options. Layout-only types like section_header, info_text, and divider do not collect data but structure the form. The order of fields in the schema is the order in which they appear on the form when a user creates a ticket from this template.

In addition to the custom fields defined in the form schema, every template can control how the standard ticket fields appear. Standard fields are: subject, description, priority, type, channel, team, assignee, and organization. For each of these, the template can set visibility (required, visible, or hidden) and default values. For example, a template might set priority to "High," type to "Service Request," team to "IT Helpdesk," and hide the channel field (defaulting to Portal). This is stored in the template's `standard_field_config`. When the ticket is created from the template, subject and description can be built from template defaults and/or from values entered in custom fields (e.g. a "Summary" custom field might map into the ticket subject). The template also stores `default_subject`, `default_priority_key`, `default_type_key`, `default_channel_key`, `default_team_id`, `default_assignee_id`, and `default_organization_id` so that even if the user does not change anything, the ticket is created with sensible defaults.

2.3 Template Approval Flow Module

A template can have an optional approval flow attached. This flow defines one or more approval steps that must be completed before the ticket is considered "approved" and can be worked on by the team. The approval flow is configured in the Template Builder: the admin defines the steps (e.g. Step 1: Manager, Step 2: Department Head), and can optionally tie each step to approval rules or to the user hierarchy (e.g. requester's manager, then requester's department head). When a user submits a ticket that was created from a template with an approval flow, the system creates the approval chain—one approval record per step—and sets the ticket's approval status to pending. The first approver receives the request; when they approve, the next step is notified, and so on until all steps are approved or someone rejects. If the template has no approval flow, the ticket is created in the normal state and does not go through approval (unless a separate approval rule applies based on priority, type, or cost). The

approval flow module is thus the “approval” part of the template: it defines who must approve this type of request and in what order.

2.4 Template Gallery and Ticket Creation from a Template

The template gallery is the list of templates presented to the user when they choose “From Template” in the New Ticket dialog. The gallery can be filtered by category and searched by name or description. Each template is shown with its name, description, category, and icon. When the user selects a template, the dialog switches to the template form: the dynamic form is rendered from the template’s `fields_schema`, and the standard fields (subject, description, priority, type, team, etc.) are shown or hidden according to `standard_field_config`. The user fills in the required and optional fields; the form is validated (required fields, types) before submit. On submit, the backend creates the ticket with the chosen defaults and the submitted values, and saves the submitted values as the “template response” linked to that ticket. If the template has an approval flow, the approval chain is created and the ticket enters the approval workflow. On the ticket detail page, agents can open a “Template response” or “Form response” view that shows the stored template response so they can see exactly what the user entered without that data being lost in the ticket body.

2.5 Duplicate Template and Usage

Admins can duplicate an existing template to create a copy with a new name; the copy keeps the same category, form schema, defaults, and approval flow, which can then be adjusted. The system also tracks `usage_count` per template (how many tickets were created from it), which helps identify the most-used request types.

3. Approval System

The approval system ensures that certain tickets are reviewed and approved by one or more people (for example a manager or department head) before the ticket is assigned to a team and worked on. This is used for high-priority requests, expensive purchases, access requests, or any scenario where policy requires managerial sign-off. The system has three main parts: approval rules (which define when approval is required and how many levels), the user hierarchy (who reports to whom), and the approval workflow (how a ticket moves through approvers and what happens when someone approves or rejects).

Approval rules are configured by admins. Each rule has a name and description and can specify conditions: which priority (e.g. High, Urgent), which type (e.g. Service Request, Change), which department, and optionally a minimum estimated cost. When a ticket is created (or when an agent explicitly sends it for approval), the engine evaluates all active rules and finds the best match. If a rule matches, the ticket is marked as requiring approval and the system builds an approval chain. The number of approval levels is set in the rule (e.g. 1 = only direct manager, 2 = manager then department head). The chain is built from the requester’s reporting hierarchy: the system looks up who the requester’s manager is, then that manager’s manager, and so on, and creates one approval record per level. Each record has an approver (user id), a

level number, and a status (pending, approved, rejected). The ticket's overall approval status is set to pending until all levels are approved or one level rejects.

From the approver's point of view, they see a "Pending approvals" list (e.g. on the Approvals page or dashboard). When they open a ticket that is pending their approval, they see the ticket details, the requester, and an approval panel showing the chain (Level 1: themselves, Level 2: next person, etc.). They can approve (with optional comments) or reject (with optional reason and comments). If they reject, the ticket's approval status is set to rejected and the requester can be notified; the ticket does not proceed to the team. If they approve, the system checks whether there are more levels: if yes, the next approver is notified and the ticket remains pending; if no, the ticket's approval status is set to approved and the ticket can then be assigned to a team and worked on normally. Thus the process is: create ticket → rule matches → chain created → first approver notified → approve → next approver notified (if any) → ... → all approved → ticket goes to team.

Agents can also send an existing ticket for approval even if it was not created with an approval rule (e.g. the situation changed and now manager sign-off is needed). In that case they use "Send for approval" and can optionally choose approvers or use a specific rule. The system supports an approvers list (users who are allowed to be chosen as approvers) so that only the right people appear in the UI. Approval history is stored per ticket so that you can see who approved or rejected at each level and when.

4. SLA Management

SLA (Service Level Agreement) management defines how quickly the organization promises to respond to and resolve tickets, and tracks whether those promises are met. The system uses two main concepts: SLA policies (the rules that define response and resolution times) and ticket SLAs (the application of those rules to individual tickets, with due dates and breach flags).

An SLA policy has a name and description and specifies two time targets: `response_minutes` (time to first response, e.g. 60 minutes) and `resolve_minutes` (time to resolution, e.g. 480 minutes). A policy can be scoped by priority (e.g. only for High and Urgent), by team (e.g. only for "IT Helpdesk"), or both; one policy can be marked as the default, used when no more specific policy matches. When a ticket is created or when its priority or team changes, the system looks up the best-matching policy (priority+team match first, then priority-only, then team-only, then default) and creates or updates a `ticket_slas` record for that ticket. This record stores the policy id, `response_due_at` (creation time + `response_minutes`), `resolve_due_at` (creation time + `resolve_minutes`), and flags `response_breached` and `resolve_breached`. So each ticket automatically gets due dates based on its priority and team.

Agents and admins can see the ticket's SLA on the ticket detail page: time remaining until response due, time remaining until resolution due, and whether either has been breached. The first public comment from an agent typically marks the response SLA as met (the system can set `first_responded_at`). Resolution is considered met when the ticket is moved to Solved or Closed. If the due time passes before that, the system can run a "check breaches" job that sets `response_breached` or `resolve_breached` and can trigger notifications. There are also actions

per ticket: pause SLA (e.g. when waiting on the customer), resume SLA, extend SLA (add more time with a reason), and reassign SLA (change the policy or due dates for that ticket).

Business hours can be defined (e.g. Monday–Friday 9–17 in a given timezone) so that SLA calculations can optionally count only working hours; the system stores schedules with day-of-week and start/end times. A separate set of SLA concepts applies to the approval process: approval SLAs track how long tickets sit in approval, with their own policies and breach checks, so that approval delays are monitored separately from resolution SLAs.

Reporting and monitoring use SLA data: “at-risk” tickets are those approaching their due dates, and reports show overall compliance (percentage of tickets that met response and resolution SLA), compliance by policy, and breach counts. This gives management visibility into whether the team is meeting its commitments.

5. Asset Management

Asset management keeps a register of IT assets—hardware and software—and links them to tickets so that support can see which device or license a request is about. Assets are used for inventory, warranty tracking, and understanding what is assigned to which user or organization.

Every asset belongs to an asset type. The system ships with types such as Computer, Mobile Device, Monitor, Printer, Network Equipment, and Software License; admins can add custom types. Each type can have a name, description, and icon. An asset record includes: a unique asset tag (e.g. LAPTOP-001), a name, the type, serial number, manufacturer, model, and status (Active, Inactive, Maintenance, Retired). The asset can be assigned to a user (assigned_to_user_id) or to an organization (assigned_to_org_id), and can have a location, purchase date, purchase cost, warranty expiry date, and notes. This supports lifecycle tracking: when a device is in maintenance or retired, status is updated rather than deleting the record.

Linking assets to tickets is a separate action: from a ticket you can “Link asset” and choose an asset from the list; from an asset you can see all tickets that have been linked to it. That way, when a user reports an issue with “my laptop,” the agent can attach the correct asset and later see the full history of tickets for that device. Asset management is typically used by agents and admins; requesters usually do not manage the asset list but may see linked assets on their tickets depending on configuration.

6. Knowledge Base

The knowledge base is a collection of articles that help users and agents find answers without opening a ticket. Articles are organized into categories and can be in draft (editing) or published (visible) state. Authors and publish dates are tracked so that content can be reviewed and kept up to date.

Categories have a name and optional description and are used to group articles (e.g. "Password Reset," "VPN," "New Starter"). An article has a title, a body (long text, often rich text), a status (draft or published), an author (user id), and an optional published_at date. Agents and admins can create and edit articles; only published articles are shown in search and browse for end users. The search function looks across article titles and body text so that users can type a question or keyword and get relevant articles. Filtering by category and status helps narrow results. The knowledge base is often used alongside the AI FAQ chat: the chat uses similar content (e.g. an FAQ document) to answer questions in conversation form, while the knowledge base is the human-readable, browsable set of articles.

7. AI FAQ Chat

The AI FAQ chat is a conversational assistant that answers questions about the Service Desk itself—how to create a ticket, how approvals work, what the dashboard shows, and so on. It is not a general-purpose chatbot; it is restricted by design to Service Desk topics so that it does not give out credentials, discuss unrelated software, or give non-Service Desk advice.

Technically, the frontend shows a floating "Have a question?" button (FaqChatBar). When the user opens it, a chat panel appears where they can type messages. Each message is sent to the backend with the current conversation history (so the AI can keep context). The backend proxies the request to an external AI service (AnythingLLM): it sends the message and history to a configured workspace (e.g. "service-desk") that has been fed with a knowledge document (e.g. AI_FAQ_KNOWLEDGE_BASE.md) containing system documentation, workflows, and FAQ content. AnythingLLM uses RAG (Retrieval-Augmented Generation) to find relevant passages and generate an answer. The answer is returned to the frontend and displayed in the chat. The user can continue the conversation or clear it to start over. Guardrails in the knowledge base instruct the AI to refuse off-topic questions (math, coding, general knowledge) and to never provide passwords or sensitive data. So the feature is "AI-powered help about the Service Desk," not a generic assistant.

8. Organizations and Teams

Organizations represent companies or business units that the Service Desk serves. Each organization has a name, optional domain (e.g. for email-based routing), industry, size, website, and notes. Users can be linked to organizations as members, and one member can be marked as the primary contact. Tickets can be associated with an organization so that agents know which company the request comes from and can filter or report by organization.

Teams are the groups of agents that handle work. A team has a name, description, and a list of members; one member can be designated as the team lead. Teams are used for assignment: a ticket can have a primary team, and agents often work from a queue of tickets assigned to their team. Multi-team support allows a ticket to have additional teams attached, with the ability to mark each team's contribution as complete or reopen it, so that cross-team tickets

are tracked clearly. Teams can optionally be linked to departments (see Departments and Hierarchy) for reporting and approval routing. Organizations and teams are managed by admins and sometimes agents (read-only); the lists are used in ticket creation, assignment, and reporting.

9. Departments and Hierarchy

Departments represent the organizational structure (e.g. IT, Finance, Operations). A department has a name, description, an optional parent department (so you can build a tree), and an optional department head (user id). This structure is used for reporting and for routing approvals: approval rules can be scoped by department, and the "department head" can be used as an approver in template or ticket approval flows.

The user hierarchy is the reporting chain: who reports to whom. It is stored as relationships between users (e.g. User A reports to User B, User B reports to User C). When an approval rule requires "manager" or "department head," the system uses this hierarchy to find the requester's manager and, if needed, the manager's manager, and creates the approval chain accordingly. Admins can set or remove a user's manager; the system can build a full chain (e.g. employee → manager → department head) for use in multi-level approvals. The hierarchy is also used to drive the org chart view, where agents and admins can see the tree of reporting relationships. So departments and hierarchy together define "how the organization is structured" and "who approves what" for the approval system.

10. Users and Authentication

Users are the people who log into the Service Desk. Each user has an email (used as login), a hashed password, full name, optional title and phone, timezone and locale, and an active flag. Users can be linked to a department and to organizations (as members). Authentication is done by email and password; on success the server issues a JWT (JSON Web Token), which the frontend sends with every subsequent request so the server knows who is logged in. The "me" endpoint returns the current user's profile and roles.

Roles determine what a user can do. The system has three main roles: Admin (full access, user management, SLA, configuration), Agent (tickets, knowledge base, assets, reports, approvals), and Requester (create and view own tickets, read knowledge base). A user can have multiple roles (e.g. both admin and agent). Role checks protect routes and API endpoints so that requesters cannot see other users' tickets or access admin pages. User management (create, edit, delete users, assign roles) is done by admins. There is also a user import: admins can download a template spreadsheet, fill in users, and upload it to create or update many users at once, which is useful for onboarding or bulk updates.

11. Reports and Analytics

Reports and analytics provide pre-built views of ticket and performance data so that managers and agents can see trends, compliance, and workload without writing queries. The exact set of reports may vary by deployment; typical ones include the following.

Ticket metrics summarize counts and times: total tickets, open vs closed, average resolution time, average first response time, often with filters by date range, team, or priority. Agent performance shows per-agent stats: how many tickets assigned, how many closed, average resolution time, and sometimes satisfaction scores, so that individual contribution is visible. SLA compliance reports show what percentage of tickets met response and resolution SLA, with breakdowns by policy, team, or period, and breach counts. Customer satisfaction (CSAT) reports aggregate ratings and comments from satisfaction surveys (e.g. 1–5 stars per ticket) so that quality of resolution can be tracked. Ticket trends show volume over time (daily, weekly, monthly) to spot busy periods and plan capacity. Team performance and department breakdown aggregate the same kinds of metrics by team or department. Approval metrics focus on the approval process: average time to approve, rejection rate, and bottlenecks (e.g. tickets stuck at a given level). Asset summary, resolution distribution (how tickets were resolved), requester activity (who submits the most tickets), hourly heatmap (when tickets arrive), and agent workload (how many open tickets per agent) provide additional operational views. An “at-risk” report lists tickets that are close to breaching SLA so that agents can prioritize. Reports can usually be filtered by date range and sometimes by team, department, or priority, and there may be an export option (e.g. CSV or Excel) for use outside the system.

12. Dashboard

The dashboard is the first page a user sees after logging in. It gives a quick overview so that agents and managers can see what needs attention without navigating to multiple screens. Typical contents include: a set of summary cards (e.g. open tickets, tickets due today, SLA at-risk count, pending approvals for the current user); a list or table of recent tickets or recent activity; and shortcuts to “My open tickets” or “Pending my approval.” The exact widgets and layout depend on the implementation; the goal is to answer “What should I look at first?” at a glance. Requesters may see a simpler dashboard (e.g. their open tickets and a link to create a new ticket). The dashboard data is loaded from a dedicated dashboard API that aggregates counts and recent items from tickets, SLA, and approvals.
